The Journal of Open Source Software

Sleep Scoring App: Visualizing and annotating rodent sleep recordings with synchronized electrophysiology, photometry, and behavior video

Yue Zhao [1¶]

1 University of Rochester, Rochester, NY, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Sleep Scoring App (`sleep_scoring`) is a local desktop application for sleep scoring rodent recordings: labeling each second as wakefulness (Wake), non-rapid-eye-movement (NREM) sleep, REM sleep, or a brief microarousal (MA), from brain electrical activity (EEG) and muscle activity (EMG). It brings EEG, EMG, an EEG spectrogram, an optional fiber-photometry norepinephrine (NE) signal, and behavior-video clips of a selected interval into one shared scoring workflow. Users can label intervals manually or start from sleep scores predicted by an automatic scoring model, then revise individual seconds or whole bouts; predicted sleep scores never overwrite the user's own labels. The default scoring model is a rule-based algorithm that adapts to expert-labeled examples: it fits the user's labels on the current recording while changing its other predictions as little as possible. Completed scores are saved back to the recording and exported as bout tables with stage and transition statistics.

Statement of need

Sleep Scoring App was developed for researchers studying how brain state regulates cerebrospinal-fluid transport during sleep, in Project 2 of the BrainFlowZZZ program at the University of Rochester. That work relates sleep states to cortical NE dynamics, which oscillate during NREM sleep and shape sleep architecture (Kjaerby et al., 2022) and may drive the vasomotion underlying glymphatic clearance (Hauglund et al., 2025). Such questions depend on brief events, such as microarousals and state transitions, that span a few seconds. Scoring them means examining the same short interval across several physiological signals and the animal's movement, and correcting boundaries across multi-hour recordings; without an integrated tool, this means moving among a signal viewer, predicted scores, and separate video software. One-second annotation is a requirement of this research, not a claim that boundaries or model estimates are accurate to one second. Sleep Scoring App is intended for experimenters scoring rodent EEG/EMG, particularly with aligned NE photometry and behavior video.

The current version takes as input a MATLAB file produced from the raw recordings by a companion preprocessing pipeline, `preprocess_sleep_data`. Other laboratories may need a thin adapter to this file layout.

State of the field

Several open tools support rodent sleep scoring. AccuSleep and its Python successor AccuSleePy (Barger et al., 2019) pair a manual-labeling interface with a neural-network scoring model. SPINDLE (Miladinović et al., 2019) provides end-to-end learned scoring across laboratories and species. Somnotate (Brodersen et al., 2024) scores with linear discriminant analysis and a

DRAFT

hidden Markov model. Visbrain Sleep (Combrisson et al., 2019) is a general polysomnography viewer and hypnogram editor.

These tools center on electrophysiology and, for most, on automatic sleep scoring, which draws on EEG, EMG, or LFP features alone. None of their scoring methods uses NE photometry, although NE dynamics have been proposed as markers for distinguishing wake, microarousals, NREM, and REM sleep (Andersen et al., 2023). Beyond automatic scoring, our work needed a workspace for labeling and correcting scores, in which NE photometry and video checks inform scoring. Project 2 previously used Viewpoint's commercial SleepScore, which did not integrate NE photometry well; that gap motivated a dedicated application. It also had to suit a large group of mostly non-programmer experimenters, yet every tool above except SPINDLE requires installing a programming environment (MATLAB or Python), and SPINDLE requires uploading recordings to its web service.

Software design

The application is designed around five tasks: (1) zero-setup installation and automatic updates, (2) visualizing and annotating at scale, (3) automatic scoring that keeps and learns from expert labels, (4) checking movement against video, and (5) saving, recovery, and export. Smooth interactive visualization is central to these tasks, so the application is built with Dash and Plotly (Plotly Technologies Inc., 2015): a local web page, shown in a native window via pywebview and backed by a Python server on the same computer.

1. Zero-setup installation and automatic updates

To make distribution easy, we packaged the Windows application with PyInstaller to install like conventional desktop software: users download a release, extract it, and double-click to open it. Our first users ran Windows; macOS support came later, and because a comparable package would need Apple's code signing and notarization, macOS users currently install from source. Packaging alone would have required a rebuild and a fresh download for every change, so we kept the application's own source code outside the packaged runtime. We further built and integrated into the application an updater, released as the reusable, dependency-free package `desktop_app_source_updater` (Zhao, 2026a), which automatically fetches and applies updates, including bug fixes, new features, and other improvements, at startup.[1]

2. Visualizing and annotating at scale

The EEG spectrogram (with a theta/delta ratio), EEG, EMG, and NE share one time axis, and the score is drawn as an overlay on every signal row (Figure 1A), so each signal can be used on its own to examine or label sleep states. We built shortcut gestures so that navigation and annotation take as few steps as possible. For example, a click selects an interval too narrow to drag at the current zoom, a right-click selects a whole contiguous bout, and a drag that reaches the edge of a zoomed-in view keeps panning, so a selection can extend arbitrarily far without losing detail. A selection is then labeled with a single key press.

In a Dash application, the interface (the embedded web page) draws the figures while the data and computation stay on the server: each interaction is sent to the server, which returns updated figure data. Plotly alone becomes unresponsive when asked to draw the millions of samples in a multi-hour recording, so the application decimates long signals on demand with Plotly Resampler (Van Der Donckt et al., 2022): the server keeps the full-resolution signals and sends the interface only enough points to draw the visible range, so zooming in or panning needs new data from the server.

[1] Updates that change the packaged dependencies still require downloading a new release.

DRAFT

Because each round trip to the server adds delay, we followed one principle for responsiveness: interactions run in the interface, and the server is asked only for signal data the interface does not yet have. Selecting and labeling therefore run in the interface. Zooming and panning must involve the server, because the refreshed traces come from there. Plotly Resampler's stock update path requests refreshed traces on every zoom or pan event, which compounds the delay during continuous movement. Users of earlier versions reported lags of one to two seconds while navigating, which disrupted scoring. We replaced that path: while the user zooms or pans continuously, the application waits for the movement to pause before requesting new traces, drops requests already overtaken by newer movement, and swaps in only the changed trace data rather than redrawing the figure.

In manual timing tests on a 2.86-hour recording with v0.17.4, the median time from a navigation request to refreshed traces on a Windows laptop was 212 ms for mouse panning and 329 ms for keyboard navigation, down roughly 65–75% from about 935 ms in an earlier version; on an Apple M4 laptop it was 64 and 181 ms. None of these is a controlled benchmark.

This interaction design is not specific to sleep; we extracted it into a runnable template for general multichannel time series (Zhao, 2026c).

3. Automatic scoring that keeps and learns from expert labels

We implemented the default scoring model as a rule-based algorithm that encodes the stage criteria used by Project 2 researchers (Kjaerby et al., 2022): it identifies Wake from EEG spectral power and REM from sustained low NE, and without usable NE it identifies only Wake and NREM. Its thresholds adapt to each recording. Each run treats the user's labels (a single labeled interval is enough) as ground truth and chooses the settings that best fit them while changing the rest of the predictions as little as possible (Figure 1). Adaptation therefore needs no multi-recording training set. Agreement with the user's labels shows only that the fit succeeded, not how accurate the predictions are elsewhere in the recording.

Alternatively, the deep-learning model sDREAMER (Chen et al., 2023), in a version adapted by the author (`sdreamer_flow`), runs when the optional PyTorch (Paszke et al., 2019) dependencies are installed; it does not adapt to user labels. With either model, the application keeps user labels separate from the predictions and places them on top, so a prediction never overwrites a label.

DRAFT

The Journal of Open Source Software

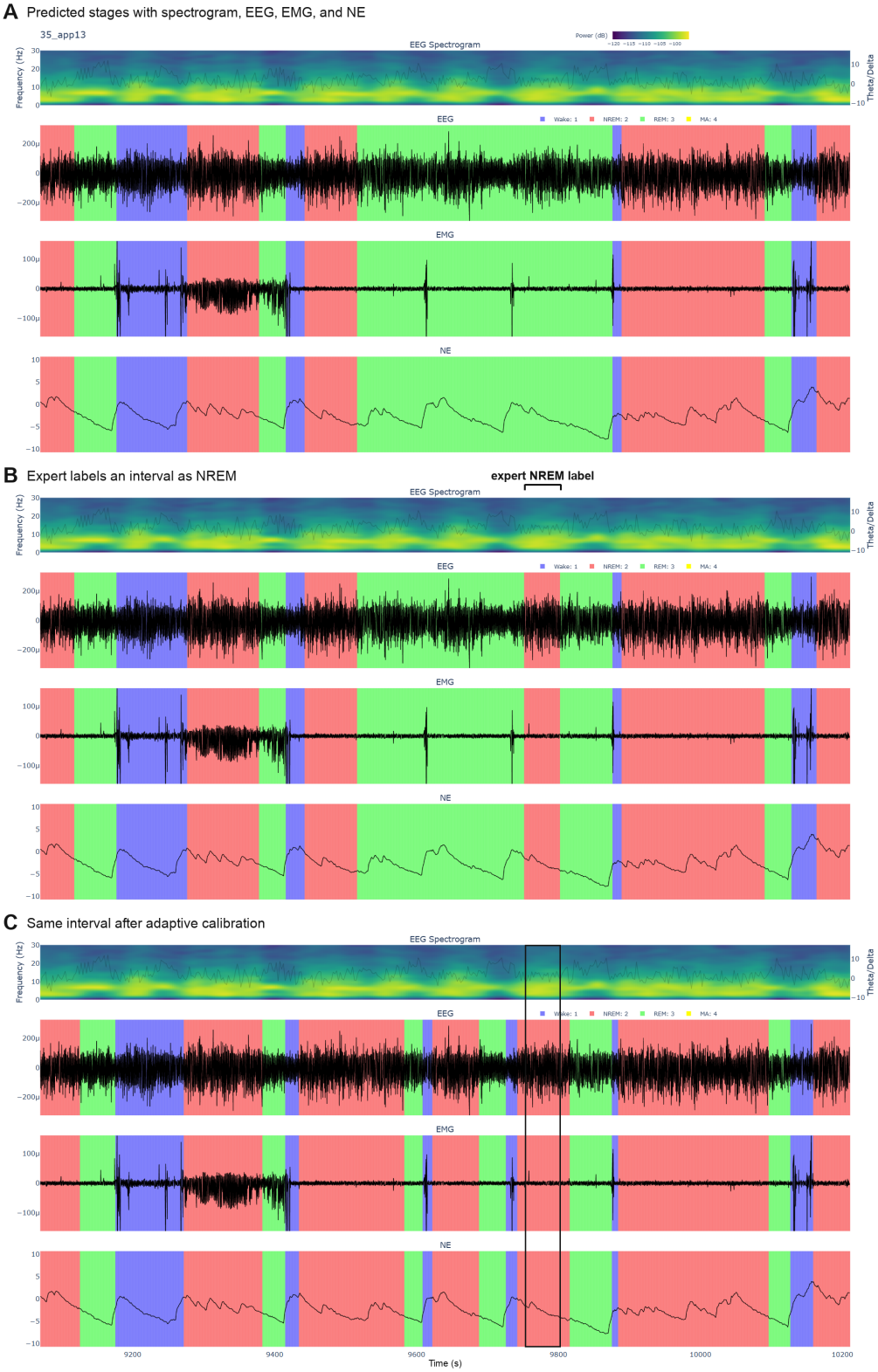


Figure 1: Expert-guided adaptation. The same interval at three successive steps: **(A)** the initial predicted stages over the spectrogram, EEG, EMG, and NE; **(B)** an expert relabels 50 s as NREM; **(C)** the model, recalibrated on 228 s of expert labels (178 s Wake outside this view, 50 s NREM), updates its predictions, including beyond the outlined interval.

4. Checking movement against video

Users sometimes check the behavior video to corroborate the movements indicated by EMG, which help distinguish sleep states. The application therefore links video to the selection: for a selected interval of up to five minutes, enough for events that span seconds and quick to extract, it cuts the matching clip with FFmpeg and plays it in the same window (Figure 2).

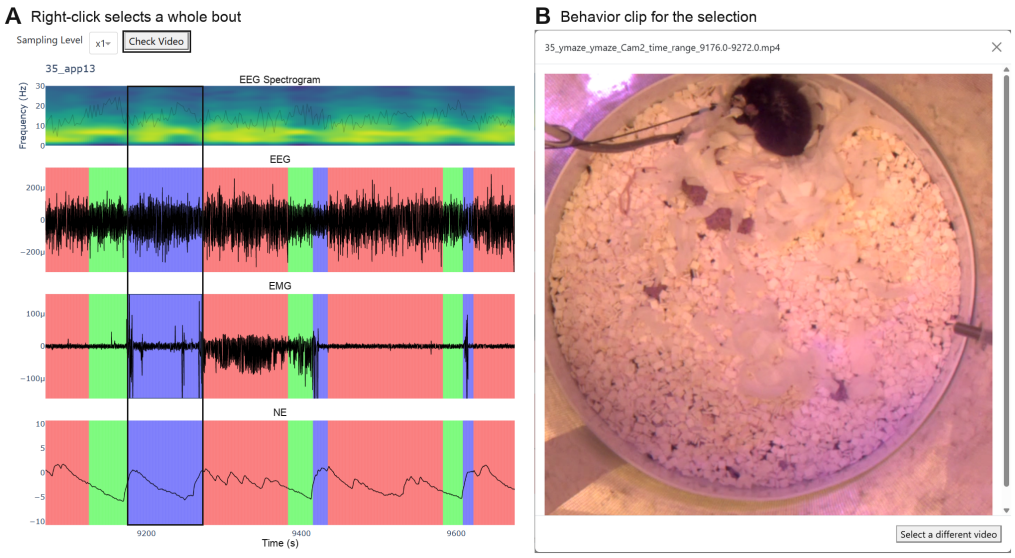


Figure 2: Selection-linked video review. (A) A right-click selects the bout at 9176–9272 s, and Check Video (outlined) opens its clip. **(B)** The 96-second clip plays beside the signals.

5. Saving, recovery, and export

Besides saving the scores back to the recording, the application can undo the most recent annotation or prediction. Saving a completely scored recording also exports bout, stage, and transition statistics to a spreadsheet; otherwise, saving reports what is left unscored, especially short gaps hard to spot when zoomed out, so the user can complete it. If the computer crashes before a save, filesystem-backed recovery restores the unsaved scores when the file is reopened.

Research impact statement

Researchers in Project 2 of the University of Rochester BrainFlowZZZ program have been the application's main users since 2023, using it to score their rodent EEG/EMG/NE recordings. The program's channel for the application has about 30 members; five users who reported their use have together scored about 1,200 hours of recordings, a lower bound on total use. Results scored with the application have been presented in conference posters. The repository provides releases, documentation, tests, demonstration videos, and archived versions (Zhao, 2026b).

AI usage disclosure

From 2023 through 2025, the author asked web-based ChatGPT and Claude coding questions (model versions not recorded). Since 2026, coding agents have assisted with code, tests, documentation, and release checks: Claude Code (Claude Opus 4.5–5.5 and Fable 5) and OpenAI Codex (GPT-5 and GPT-6), with sessions recorded in the repository's work log (`treaty_docs/`), kept under the author's Agent Collab Treaty conventions for human–agent collaboration. Claude Code and Codex also helped plan, draft, and review this manuscript.

DRAFT

The author made the architectural and design decisions, reviewed and tested all AI-assisted code before release, and reviewed, revised, and verified the manuscript against the software and cited sources.

Acknowledgments

This work was supported by the BRAIN Initiative of the US National Institutes of Health (U19NS128613).

We thank Mie Andersen, Jingyuan Chen, Natalie Linea Hauglund, Shadi Sartipi, Anastasia Tsopanidou, Verena Untiet, and Yuan Yao for helping define the sleep-stage rules that the rule-based model encodes. We especially thank Mie Andersen, Rebeka Kovács, Klaudia Anna Tokarska, and Anastasia Tsopanidou, who were among the application's earliest users and gave extensive one-on-one feedback. We also thank the other users whose feedback and feature requests shaped the application: at the University of Copenhagen, Katharina Franziska Baumgart, Antonia Benita Beiersdorfer, Julia Johanna Berger, Zuzanna Bojarowska, Saiyue Deng, Stefanie Gregoriades, Natalie Linea Hauglund, Jaspreet Kaur, Elise Schiøler Nielsen, Tessa Radovanovic, Laurence Vince Riesselmann, Katia Soud, and Yingshan Wang; and at the University of Rochester, Ziru Deng, Tianyi Tong, and Jianbo Wang. We thank Douglas Kelley for supervising this work, Verena Untiet for leading the Copenhagen group, and Maiken Nedergaard for leading the BrainFlowZZZ program.

References

Andersen, M., Tsopanidou, A., Radovanovic, T., Compere, V., Hauglund, N., Nedergaard, M., & Kjaerby, C. (2023). Using fiber photometry in mice to estimate fluorescent biosensor levels during sleep. *Bio-Protocol*, *13*(15), e4734. https://doi.org/10.21769/BioProtoc.4734

Barger, Z., Frye, C. G., Liu, D., Dan, Y., & Bouchard, K. E. (2019). Robust, automated sleep scoring by a compact neural network with distributional shift correction. *PLOS ONE*, *14*(12), e0224642. https://doi.org/10.1371/journal.pone.0224642

Brodersen, P. J. N., Alfonsa, H., Krone, L. B., Blanco-Duque, C., Fisk, A. S., Flaherty, S. J., Guillaumin, M. C. C., Huang, Y.-G., Kahn, M. C., McKillop, L. E., Milinski, L., Taylor, L., Thomas, C. W., Yamagata, T., Foster, R. G., Vyazovskiy, V. V., & Akerman, C. J. (2024). Somnotate: A probabilistic sleep stage classifier for studying vigilance state transitions. *PLOS Computational Biology*, *20*(1), e1011793. https://doi.org/10.1371/journal.pcbi.1011793

Chen, J., Yao, Y., Anderson, M., Hauglund, N., Kjaerby, C., Untiet, V., Nedergaard, M., & Luo, J. (2023). sDREAMER: Self-distilled mixture-of-modality-experts transformer for automatic sleep staging. *2023 IEEE International Conference on Digital Health (ICDH)*, 131–142. https://doi.org/10.1109/ICDH60066.2023.00028

Combrisson, E., Vallat, R., O'Reilly, C., Jas, M., Pascarella, A., Saive, A.-L., Thiery, T., Meunier, D., Altukhov, D., Lajnef, T., Ruby, P., Guillot, A., & Jerbi, K. (2019). Visbrain: A multi-purpose GPU-accelerated open-source suite for multimodal brain data visualization. *Frontiers in Neuroinformatics*, *13*, 14. https://doi.org/10.3389/fninf.2019.00014

Hauglund, N. L., Andersen, M., Tokarska, K., Radovanovic, T., Kjaerby, C., Sørensen, F. L., Bojarowska, Z., Untiet, V., Ballestero, S. B., Kolmos, M. G., Weikop, P., Hirase, H., & Nedergaard, M. (2025). Norepinephrine-mediated slow vasomotion drives glymphatic clearance during sleep. *Cell*, *188*(3), 606–622.e17. https://doi.org/10.1016/j.cell.2024.11.027

Kjaerby, C., Andersen, M., Hauglund, N., Untiet, V., Dall, C., Sigurdsson, B., Ding, F., Feng, J., Li, Y., Weikop, P., Hirase, H., & Nedergaard, M. (2022). Memory-enhancing properties

DRAFT

of sleep depend on the oscillatory amplitude of norepinephrine. *Nature Neuroscience*, *25*(8), 1059–1070. https://doi.org/10.1038/s41593-022-01102-9

Miladinović, Đ., Muheim, C., Bauer, S., Spinnler, A., Noain, D., Bandarabadi, M., Gallusser, B., Krummenacher, G., Baumann, C. R., Adamantidis, A., & others. (2019). SPINDLE: End-to-end learning from EEG/EMG to extrapolate animal sleep scoring across experimental settings, labs and species. *PLOS Computational Biology*, *15*(4), e1006968. https://doi.org/10.1371/journal.pcbi.1006968

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems 32*, 8024–8035. https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

Plotly Technologies Inc. (2015). *Collaborative data science*. Plotly Technologies Inc. https://plot.ly

Van Der Donckt, J., Van Der Donckt, J., Deprost, E., & Van Hoecke, S. (2022). plotly-resampler: Effective visual analytics for large time series. *2022 IEEE Visualization and Visual Analytics (VIS)*, 21–25. https://doi.org/10.1109/VIS54862.2022.00013

Zhao, Y. (2026a). *desktop_app_source_updater: A dependency-free source updater for packaged python desktop applications*. Zenodo. https://doi.org/10.5281/zenodo.21763329

Zhao, Y. (2026b). *Sleep Scoring App: Interactive review and correction of rodent sleep annotations with synchronized electrophysiology, photometry, and behavior video*. Zenodo. https://doi.org/10.5281/zenodo.21748494

Zhao, Y. (2026c). *timeseries_app_cookbook: A runnable template and recipe book for building interactive desktop apps that navigate and annotate long time series*. Zenodo. https://doi.org/10.5281/zenodo.21763327

DRAFT